

AssetFlow

Smart Asset Management & Resource Allocation Platform

IIT Roorkee Cultural Council · Full-Stack Engineering Challenge

1. Problem Understanding

The IIT Roorkee Cultural Council manages a large pool of shared resources — DSLR cameras, studio lighting, audio systems, costumes, stage props, recording equipment, and event infrastructure — across multiple sections and events. Currently this is coordinated through spreadsheets, registers, and informal messaging, which frequently leads to double-bookings, lost accountability, and assets not being returned on time.

AssetFlow is a full-stack web platform that digitizes this entire workflow: a centralized inventory catalogue, a self-service booking system for users, an approval workflow for administrators, automated tracking of issue/return cycles with due dates, and an analytics dashboard giving the council real-time visibility into utilization and overdue items.

Core goals

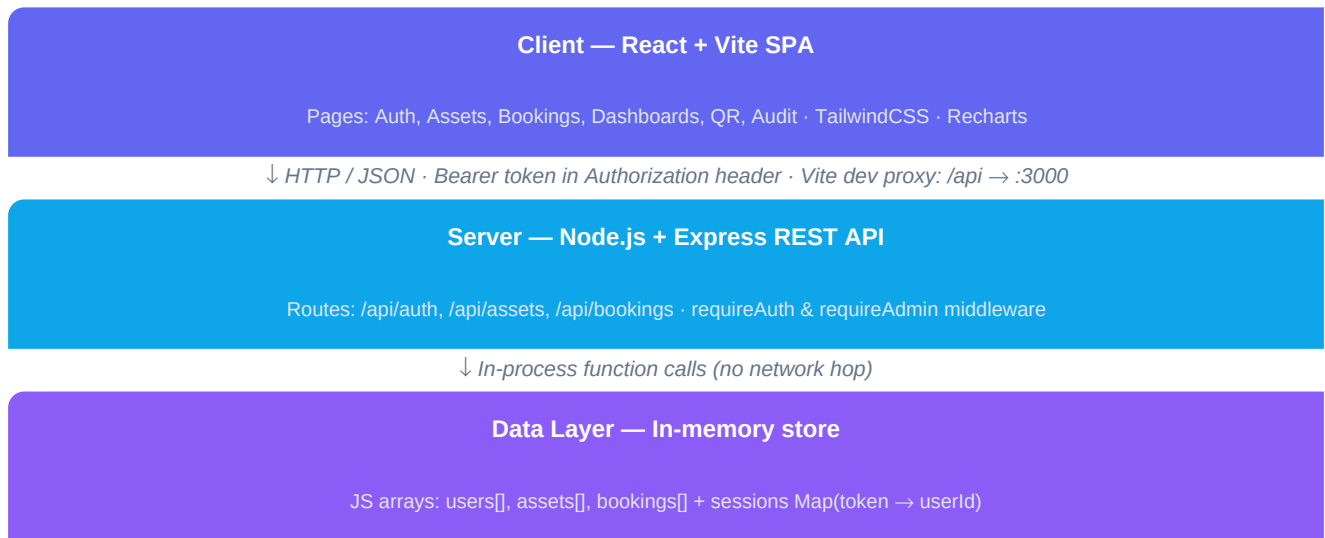
- Single source of truth for inventory — quantities, categories, condition, and live availability.
- Self-service booking for students/section heads, replacing manual WhatsApp/Excel coordination.
- Admin approval workflow that prevents overbooking and double allocation automatically.
- Full lifecycle tracking — request → approval → issue → return → audit log.
- Operational visibility through role-specific dashboards (utilization, overdue, top-requested assets).
- Bonus operational tooling: QR-based issue/return and in-app notifications for due dates.

Actors & roles

Role	Capabilities
User (USER)	Register/login, browse & search assets, submit booking requests, track request status, view personal borrowing history, receive notifications.
Admin (ADMIN)	All user capabilities plus: full asset CRUD, approve/reject bookings, issue & return assets, view system-wide activity, audit logs, QR generation/scanning, analytics dashboard.

2. System Architecture

AssetFlow follows a classic three-tier client-server architecture, kept deliberately simple to ship within the hackathon timeline while remaining easy to extend to a persistent database (see Section 6).



Note: the data layer is intentionally an in-memory store for the hackathon — this keeps the demo dependency-free (no DB setup needed for judges) at the cost of resetting on server restart. The access pattern is already structured so each array can be swapped for a Prisma/PostgreSQL model with minimal route changes (see Section 6, Future Improvements).

Request lifecycle — example: booking approval

1. Admin clicks Approve on a PENDING booking in the React admin panel.
2. Frontend sends PATCH /api/bookings/:id/approve with the JWT-style session token in the Authorization header.
3. requireAuth middleware resolves the token via the sessions Map to a user object; requireAdmin checks role === 'ADMIN'.
4. updateBookingStatus() mutates the booking object in-place, setting status to APPROVED.
5. Server responds with the serialized booking (joined with user + asset summary).
6. Frontend refetches the bookings list; the row moves from the Pending tab to the Approved tab.

Authentication model

AssetFlow uses a simplified bearer-token session model rather than full JWT: on login/register, the server generates a random opaque token and stores a token → userId mapping in an in-memory Map (sessions). The frontend stores this token in localStorage and attaches it as Authorization: Bearer <token> on every request via an axios interceptor. requireAuth resolves the token back to a user; requireAdmin additionally checks role === 'ADMIN'. Passwords are currently stored in plaintext in the demo seed data — Section 6 covers the production hardening path (bcrypt + real JWT).

3. Database Schema & ERD

The current implementation models three core entities as in-memory arrays: **User**, **Asset**, and **Booking**. A fourth derived concept, **Notification**, is computed on the frontend for the bonus feature. The schema below reflects the exact shapes used in `backend/src/server.js` and maps directly onto a future Prisma schema.

User		
id	string	Primary key. Random ID generated via timestamp + random string.
name	string	Display name shown across UI and audit logs.
email	string	Unique, case-insensitive. Used for login.
password	string	Plaintext in demo (see Future Improvements for bcrypt hashing).
role	'USER' 'ADMIN'	Determines UI routing and gates admin-only endpoints.

Asset		
id	string	Primary key.
name	string	e.g. 'DSLR Canon EOS 5D Mark IV'.
category	string	Camera, Audio, Lighting, Costume, Props, Recording, Infrastructure.
description	string	Free-text description shown on asset cards.
quantity	number	Total owned units (not currently available units).
status	enum	AVAILABLE IN_USE MAINTENANCE DAMAGED.
condition	string	Excellent Good Fair Poor.
bookedQuantity*	number (derived)	*Not stored — computed on read as sum of quantities across active bookings (PENDING/APPROVED/ISSUED) for this asset.

Booking		
id	string	Primary key.
userId	string (FK → User.id)	The requester.
assetId	string (FK → Asset.id)	The requested asset.
quantity	number	Units requested; validated against availability at creation.
startDate	date string	Requested loan start date (YYYY-MM-DD).
endDate	date string	Requested loan end date.
dueDate	date string	Currently mirrors endDate; used for overdue checks.
purpose	string	Free-text reason / event name.
status	enum	PENDING APPROVED REJECTED ISSUED RETURNED.
rejectionReason*	string (optional)	Set when status becomes REJECTED.
returnedAt*	date string (optional)	Set when status becomes RETURNED.

Entity relationships



USER (1) — (N) BOOKING: a user can have many bookings; each booking belongs to exactly one user via `userId`. **ASSET (1) — (N) BOOKING:** an asset can appear in many bookings over time; each booking references exactly one asset via `assetId`.

4. API Overview — Auth & Assets

All endpoints are prefixed with `/api`. Protected endpoints require Authorization: Bearer <token>; admin-only endpoints additionally require role `=== 'ADMIN'`.

Auth endpoints			
POST	/api/auth/register	—	Create account. Body: {name, email, password, role?}. Rejects duplicate email (case-insensitive). Returns {token, user}.
POST	/api/auth/login	—	Body: {email, password}. Validates against seeded/registered users. Returns {token, user}.
GET	/api/auth/me	Auth	Returns the currently authenticated user, resolved from the bearer token.
Asset endpoints			
GET	/api/assets	—	List all assets. Each asset is serialized with a computed bookedQuantity field.
GET	/api/assets/:id	—	Get a single asset by ID, with bookedQuantity. 404 if not found.
POST	/api/assets	Admin	Create asset. Body: {name, category, description?, quantity?, status?, condition?}. name & category required.
PUT	/api/assets/:id	Admin	Partial update via Object.assign. quantity is coerced to a number if provided.
DELETE	/api/assets/:id	Admin	Removes the asset from the array. Returns 204 No Content.

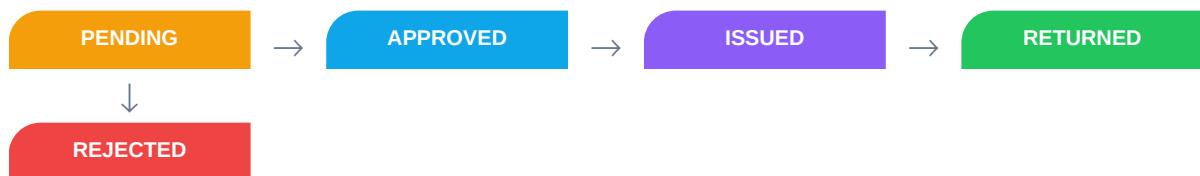
Key business rule — live availability

Rather than storing a separate `availableQuantity` field that could drift out of sync, `AssetFlow` computes availability on every read via `bookedQuantityForAsset(assetId)`. This sums the quantity of every booking for that asset whose status is in `{PENDING, APPROVED, ISSUED}` — i.e. any booking that currently has a claim on stock, whether or not it has physically left the shelf yet. `available = asset.quantity - bookedQuantity`. This single source of truth guarantees the frontend never shows stale availability and is the basis for the overbooking guard described in Section 5.

5. API Overview — Bookings & Lifecycle

Booking endpoints			
GET	/api/bookings	Auth	Admin sees all bookings; regular users see only their own (filtered by userId).
GET	/api/bookings/mine	Auth	Always returns only the authenticated user's bookings, regardless of role.
POST	/api/bookings	Auth	Create a request. Body: {assetId, quantity, startDate, endDate, purpose}. Validates $1 \leq \text{quantity} \leq \text{available}$.
PATCH	/api/bookings/:id/approve	Admin	PENDING → APPROVED.
PATCH	/api/bookings/:id/reject	Admin	PENDING → REJECTED. Optional body: {reason}, stored as rejectionReason.
PATCH	/api/bookings/:id/issue	Admin	APPROVED → ISSUED. Physical handover to the user.
PATCH	/api/bookings/:id/return	Admin	ISSUED → RETURNED. Sets returnedAt to today's date.

Booking status state machine



Every transition is admin-triggered via a dedicated PATCH endpoint (Section above), keeping the state machine linear and auditable: a booking can only move forward (or branch once into REJECTED from PENDING). There is deliberately no "un-approve" or "un-issue" — corrections are handled by creating a new booking, which keeps the audit trail honest.

Overbooking guard (validated on creation)

When a user submits `POST /api/bookings`, the server computes `availableQuantity = asset.quantity - bookedQuantityForAsset(asset.id)` *before* inserting the new booking, and rejects the request with HTTP 400 if `requestedQuantity > availableQuantity` (or < 1). Because `bookedQuantityForAsset` already counts PENDING and APPROVED bookings (not just ISSUED), two users cannot simultaneously over-claim the same stock even before either request is approved — the second request is rejected at submission time with a clear message: *"Quantity must be between 1 and N"*.

Response shape — `serializeBooking()`

Every booking response is enriched server-side so the frontend never needs a second round-trip: user is replaced with a safe public profile (id, name, email, role — password stripped), and asset is replaced with a lightweight summary (id, name, category). This keeps booking list/table renders a single fetch.

6. Design Decisions & Trade-offs

In-memory data store vs. PostgreSQL

Chosen: plain JS arrays + Map.

Zero setup for judges — clone, npm install, npm run dev, done. No DB driver, migration, or seed script can fail on an unfamiliar machine. Trade-off: data resets on restart (mitigated by seeding 3 demo accounts + sample assets/bookings on boot, documented in README). The route layer already does ID-based lookups and array filters that map 1:1 onto Prisma's findUnique/findMany/where clauses — migration is mechanical, not a redesign.

Computed availability vs. stored counter

Chosen: derive bookedQuantity on every read.

A stored 'available' counter that's incremented/decremented across 5 different status transitions is a classic source of drift bugs (forget one decrement on reject-after-approve, and inventory permanently understates). Deriving it fresh from the bookings array on every GET is O(bookings) per asset — trivial at hackathon scale — and is provably always correct.

Opaque session tokens vs. JWT

Chosen: random token + server-side session Map.

Functionally equivalent to JWT for a single-server demo (stateless vs. stateful is irrelevant with one process), but avoids pulling in jsonwebtoken + signing-secret management for a 48-hour build. The axios interceptor / Authorization header pattern is identical either way, so swapping to real JWT later (Section 8) only touches the auth route handlers, not the frontend.

Role-based access via two middlewares

Chosen: requireAuth then requireAdmin, composed per-route.

Express middleware composition (app.post(path, requireAuth, requireAdmin, handler)) makes the access policy visible at the route declaration — no need to open the handler body to know who can call it. This also means public endpoints (asset browsing) simply omit both middlewares.

Status enum as flat strings vs. nested objects

Chosen: single status: 'PENDING' | 'APPROVED' | ... field.

A flat enum makes the state machine (Section 5) trivial to query, filter, and badge in the UI (tab counts are a one-line .filter()). Auxiliary fields (rejectionReason, returnedAt) are only added when the relevant transition occurs, keeping normal-path objects small.

7. Frontend Architecture & Tech Stack

Layer	Technology	Notes
Build tool	Vite 8	Sub-second HMR; dev server proxies /api to Express on :3000.
UI framework	React 19	Functional components + hooks throughout, no class components.
Styling	TailwindCSS 4	Utility-first; consistent spacing/color scale across 9 pages.
Routing	React Router v7	Nested routes with RequireAuth / RequireAdmin guard components.
HTTP client	Axios	Single instance with request (attach token) and response (401 → logout) interceptors.
Charts	Recharts	Bar (utilization), Pie (category split), Line (monthly trend).
Icons	Lucide React	Consistent 15–22px icon set across sidebar, buttons, tables.
Toasts	React Hot Toast	Success/error feedback for every mutation (approve, issue, etc.).
QR	qrcode + html5-qrcode	Client-side QR generation (PNG download) and camera-based scanning.
Backend	Node.js + Express	13 REST routes, in-memory store, see Sections 4–5.

Page-level breakdown

Route	Page	Role	Purpose
/login, /register	AuthPages	Public	Login & registration with client-side validation, role selector.
/dashboard	UserDashboard	User	Stat cards (active/pending/overdue/returned), active loans table, overdue banner.
/assets	AssetsPage	User	Searchable, category-filterable asset grid + booking request modal with date validation.
/bookings	MyBookingsPage	User	Tabbed view: active / pending / rejected, with overdue highlighting.
/history	HistoryPage	User	Searchable table of RETURNED bookings (personal borrowing history).
/admin/dashboard	AdminDashboard	Admin	6 stat cards + bar/pie/line charts + top-5 most-requested assets.
/admin/assets	AdminAssets	Admin	Full asset CRUD table with add/edit modal and delete confirmation.
/admin/bookings	AdminBookings	Admin	Tabbed approval queue: approve/reject/issue/return actions.
/admin/history	AdminHistory	Admin	System-wide RETURNED bookings across all users.
/admin/audit	AuditLogsPage	Admin	Searchable, filterable activity feed with action-type badges.
/admin/qr	QRPage	Admin	Generate per-asset QR (download PNG) and scan to issue/return.

8. Future Improvements

Persistent database (PostgreSQL + Prisma)

Replace the three in-memory arrays with Prisma models matching Section 3's schema exactly. Each route's `array.find/filter` becomes `prisma.model.findUnique/findMany` — the `serializeAsset/serializeBooking` shape stays identical, so the frontend requires zero changes.

Real JWT + bcrypt password hashing

Sign tokens with `jsonwebtoken` (HS256, expiry claim) and hash passwords with `bcrypt` at registration. `requireAuth` verifies the signature instead of a Map lookup — enables horizontal scaling across multiple server instances without shared session state.

Email notifications

Extend the in-app notification bell with actual emails (e.g. via `Nodemailer/Resend`) for booking approvals, rejections, and due-date reminders 24h before `dueDate`.

Scheduled overdue sweep

A daily cron job that scans bookings where `status === 'ISSUED' && dueDate < today`, flags them as overdue, and triggers reminder notifications — currently overdue detection is computed on-the-fly in the UI only.

Audit log persistence

Wire every state-changing route (asset CRUD, all booking transitions) to append an `AuditLog` row capturing actor, action, target, and timestamp — the `AuditLogsPage` UI already exists and expects exactly this shape.

Multi-organization support

Add an `organizationId` foreign key to `User` and `Asset`, scoping all queries — lets the same platform serve multiple college clubs/departments with isolated inventories.

Dockerized deployment

`docker-compose.yml` with three services (frontend, backend, postgres) for one-command, judge-reproducible setup that also matches production deployment topology.

Mobile application

The REST API is already platform-agnostic; a React Native or Flutter client can reuse the entire backend unchanged, with the QR scanner becoming the primary issue/return interface for on-the-ground volunteers.

Conclusion

AssetFlow delivers the complete mandatory feature set — authentication with role-based access, full asset inventory management, a self-service booking flow with a server-enforced overbooking guard, a linear admin approval-to-return workflow, role-specific analytics dashboards, and personal/system-wide borrowing history — alongside three bonus features: in-app notifications, audit logging, and QR-based issue/return operations. The architecture prioritizes a frictionless judge experience (single in-memory store, one command to run both servers, seeded demo accounts) while keeping every data-access pattern shaped for a direct, low-risk migration to PostgreSQL and JWT in a production setting.